

Performance : k6 côté serveur, Lighthouse côté navigateur

`PERF.md` mesure la performance à deux endroits qui ne répondent pas à la même question. **k6** charge l'API et dit combien de temps elle met à répondre sous charge. **Lighthouse** audite une page dans un navigateur émulé et dit ce que ressent l'utilisateur au chargement. L'enjeu à l'oral : un score Lighthouse dépend **de la configuration mesurée** autant que du code, d'où l'écart 56 → 77 → 99 sur la même application.

k6 : charge sur les endpoints critiques

k6 est un outil de test de charge scriptable en JavaScript (`perf/k6/*.js`). Un **VU** (Virtual User) est un utilisateur simulé ; le scénario `ramping-vus` monte progressivement leur nombre.

```
scenarios: { ramp: { executor: 'ramping-vus', startVUs: 0,
  stages: [ { duration: '15s', target: 5 }, { duration: '30s', target: 5 }, { duration: '5s', target: 0 } ] } },
thresholds: { http_req_failed: ['rate<0.01'], http_req_duration: ['p(95)<1500'] },
```

Les **seuils** (`thresholds`) font échouer le run s'ils sont dépassés. **p95** = 95 % des requêtes sont plus rapides que cette valeur : plus parlant qu'une moyenne, qui masque les lenteurs.

Endpoint	Pourquoi il est critique	p95 mesuré	Seuil	Erreurs
POST <code>/api/files</code>	le plus lourd (réception + écriture stockage)	385,64 ms	< 3000 ms	0 %
GET <code>/api/d/{token}</code>	le plus sollicité (chaque destinataire)	556,42 ms	< 1500 ms	0 %
GET <code>/api/ping</code>	référence à vide	11 ms	< 300 ms	0,01 %

Points à savoir dire :

- Charge modeste (5 VUs pour upload/download, 10 pour ping) : les résultats prouvent l'absence de problème à ce niveau, pas la tenue à forte charge. `PERF.md` le dit : goulot « non identifié ».
- Le download est plus lent que l'upload (556 vs 386 ms) : écart signalé **à comprendre**, non expliqué à ce jour.
- k6 tourne **hors CI** (une charge sur un runner partagé fait du bruit) : lancé à la main.

Lighthouse : trois configurations, trois scores

Lighthouse (Google) note Performance, Accessibilité, Bonnes pratiques et SEO (Search Engine Optimization), avec des Core Web Vitals : FCP (First Contentful Paint), LCP (Largest Contentful Paint), TBT (Total Blocking Time), CLS (Cumulative Layout Shift). Mêmes conditions pour les trois runs : page `/upload`, mobile émulé.

Métrique	Debug	ng serve -- configuration production	nginx + build prod réel (Docker)
Performance	56	77	99
FCP	3,2 s	1,9 s	1,3 s
LCP	6,6 s	2,5 s	2,2 s
TBT	100 ms	30 ms	10 ms
CLS	0,052	0,024	0

Pourquoi cet écart est significatif à l'oral

- **Ce n'est pas le code qui change, c'est la façon de le servir.** Même application, trois déploiements. 56 → 77 : retirer le mode debug (bundle non minifié). 77 → 99 : un vrai `ng build` de production servi par nginx (minification, compression HTTP) au lieu du serveur de développement.
- Poids : le run dev-server estimait environ 1701 Kio de JS à économiser ; le run nginx transfère 123 KiB au total.
- **Le run le plus représentatif est le 99**, seul à correspondre à la chaîne de déploiement réelle (`frontend/Dockerfile` + nginx). Le 56 est une capture de debug, non représentative.
- Une hypothèse posée avant mesure (le `ng serve` prod pénalise, nginx fera mieux) a été **confirmée par mesure** : c'est la démarche à raconter.
- Honnêteté : les deux premiers runs datent du 22/09, avant plusieurs PR touchant `/upload` ; `PERF.md` les qualifie d'**indicatifs**.

Un score élevé n'est pas tout : la régression SEO

Le run à 99 a un SEO de 82 (contre 91) : le fallback SPA (Single Page Application) de nginx renvoie `index.html` en 200 sur `/robots.txt`, que Lighthouse tente de lire comme un robots.txt (13 erreurs). Il manque aussi une meta description. Correctifs prévus, non faits. Les en-têtes de sécurité manquants (CSP, HSTS) ne sont pas comptés dans le score.

`/d/:token` : un CLS de 0,076

Sur la page de téléchargement (Performance 97), l'appel `GET /api/d/{token}` n'est lancé qu'**après** le chargement du bundle JS, donc la bannière et le champ mot de passe apparaissent tard et décalent la mise en page. Cause « probable » selon `PERF.md`, pas prouvée. Piste : lancer l'appel en parallèle du démarrage d'Angular.

Budgets front

`frontend/angular.json` déclare un budget : bundle initial warning 500 ko, error 1 Mo ; style par composant warning 6 ko, error 10 ko. Le job `assert-prod-bundle` échoue au-delà de l'error. Mesure du 28/09 : 281,74 ko brut, 76,76 ko transférés (gzip).

Dans DataShare

- Méthodologie, résultats, interprétation : `PERF.md` §1 à §6.
- Scripts : `perf/k6/` (`ping-smoke.js`, `upload.js`, `download.js`).
- Budgets : `frontend/angular.json` ; garde CI : job `assert-prod-bundle` (`docs/CI.md`).
- Captures : `docs/screenshots/` (k6, Lighthouse upload/download).

À retenir

1. k6 mesure l'API sous charge (p95, taux d'erreur, seuils) ; Lighthouse mesure l'expérience de chargement dans un navigateur. Deux outils, deux questions.
2. Les p95 sont bons (386 ms upload, 556 ms download, 0 % d'erreur) mais à faible charge : ils ne prouvent pas la tenue à grande échelle.
3. 56 → 77 → 99 : même code, trois manières de le servir. Le score dépend de la configuration, donc seul le run nginx + build prod réel représente la production.
4. Hypothèse posée puis confirmée par mesure, et limites déclarées (captures anciennes, SEO 82, CLS de `/d/:token`) : `PERF.md` ne cache pas ses points faibles.